

Laboratório 5

Letícia da Costa Venancio - 250125

Carregando pacotes necessários

```
library(tidyverse)
library(leaflet)
```

Criando a função analisa_aeronave

```
analisa_aeronave <- function(tail_number, arquivo){

  ## função que filtra apenas as linhas onde
  ## TAIL_NUMBER corresponde à aeronave de interesse
  filtra_voos_aeronave <- function(chunk_dados, indice_posicao) {
    chunk_dados %>%
      filter(TAIL_NUMBER == tail_number)
  }

  ## lendo o arquivo em blocos de 100.000 linhas por vez
  tabela_voos <- read_csv_chunked(
    file = arquivo,
    callback = DataFrameCallback$new(filtra_voos_aeronave),
    chunk_size = 100000,
    show_col_types = FALSE)

  ## organizando datas e horários
  tabela_voos_organizada <- tabela_voos %>%

  # tirando voos sem horário de partida/tempo no ar
  filter(!is.na(DEPARTURE_TIME), !is.na(AIR_TIME)) %>%
```

```

mutate(
  # como o tempo está no formato 1430 para 14:30,
  # adiciona-se zeros à esquerda para ter sempre 4 dígitos
  horario_str = str_pad(DEPARTURE_TIME, width = 4, pad = "0"),
  hora = substr(horario_str, 1, 2),
  minuto = substr(horario_str, 3, 4),

  # criando uma coluna Date-Time para poder ordenar cronologicamente
  data_hora_exata = make_datetime(YEAR, MONTH, DAY,
                                   as.numeric(hora), as.numeric(minuto)),

  # velocidade média do voo em milhas por hora (mph)
  velocidade_media_mph = (DISTANCE / AIR_TIME) * 60,

  ORIGIN_AIRPORT = as.character(ORIGIN_AIRPORT),
  DESTINATION_AIRPORT = as.character(DESTINATION_AIRPORT)
) %>%
# ordenando de forma linear no tempo
arrange(data_hora_exata)

## utilizar as coordenadas exatas dos aeroportos envolvidos
## (presentes no arquivo airports)
caminho_aeroportos <- "airports.csv"
tabela_aeroportos <- read_csv(caminho_aeroportos, show_col_types = FALSE) %>%
  mutate(IATA_CODE = as.character(IATA_CODE))

# juntar origem e destino para obter latitudes e longitudes
tabela_mapa <- tabela_voos_organizada %>%
  left_join(tabela_aeroportos,
            by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
  rename(lat_origem = LATITUDE,
         long_origem = LONGITUDE, nome_origem = AIRPORT) %>%
  left_join(tabela_aeroportos,
            by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
  rename(lat_destino = LATITUDE,
         long_destino = LONGITUDE, nome_destino = AIRPORT) %>%
  # remove na's
  filter(!is.na(lat_origem) & !is.na(long_origem) &
         !is.na(lat_destino) & !is.na(long_destino))

## produzindo o mapa
mapa_trajetos <- leaflet() %>%

```

```

addTiles()

# trajeto sequencial da aeronave
# a espessura da linha (weight) reflete a velocidade média do voo
for(i in 1:nrow(tabela_mapa)) {
  voo_atual <- tabela_mapa[i, ]

  espessura <- case_when(
    voo_atual$velocidade_media_mph < 300 ~ 1.5,
    voo_atual$velocidade_media_mph >= 300 &
      voo_atual$velocidade_media_mph < 450 ~ 3,
    voo_atual$velocidade_media_mph >= 450 ~ 5,
    TRUE ~ 2
  )

  mapa_trajetos <- mapa_trajetos %>%
    addPolylines(
      lng = c(voo_atual$long_origem, voo_atual$long_destino),
      lat = c(voo_atual$lat_origem, voo_atual$lat_destino),
      weight = espessura,
      color = "#2c3e50",
      opacity = 0.5,
      popup = paste(
        "<b>Voo:</b>", voo_atual$FLIGHT_NUMBER, "<br>",
        "<b>Data/Hora:</b>", voo_atual$data_hora_exata, "<br>",
        "<b>Origem:</b>", voo_atual$nome_origem, "<br>",
        "<b>Destino:</b>", voo_atual$nome_destino, "<br>",
        "<b>Velocidade Média:</b>", round(voo_atual$velocidade_media_mph, 1),
        "mph"
      )
    )
}

# adicionando marcadores para os aeroportos visitados pela aeronave
aeroportos_visitados <- bind_rows(
  tabela_mapa %>%
    select(lng = long_origem, lat = lat_origem, nome = nome_origem),
  tabela_mapa %>%
    select(lng = long_destino, lat = lat_destino, nome = nome_destino)
) %>% distinct()

mapa_trajetos <- mapa_trajetos %>%

```

```

    addCircleMarkers(
      data = aeroportos_visitados,
      lng = ~lng, lat = ~lat,
      radius = 4, color = "#e74c3c", stroke = FALSE, fillOpacity = 0.8,
      popup = ~nome
    )

# retorna a tabela em formato tidy (com todas as colunas do arquivo flights)
# e o gráfico de trajetos
retorno_final <- list(
  tabela = tabela_voos_organizada %>%
    select(-horario_str, -hora, -minuto, -data_hora_exata, -velocidade_media_mph),
  grafico = mapa_trajetos
)

return(retorno_final)
}

```

Utilizando a função

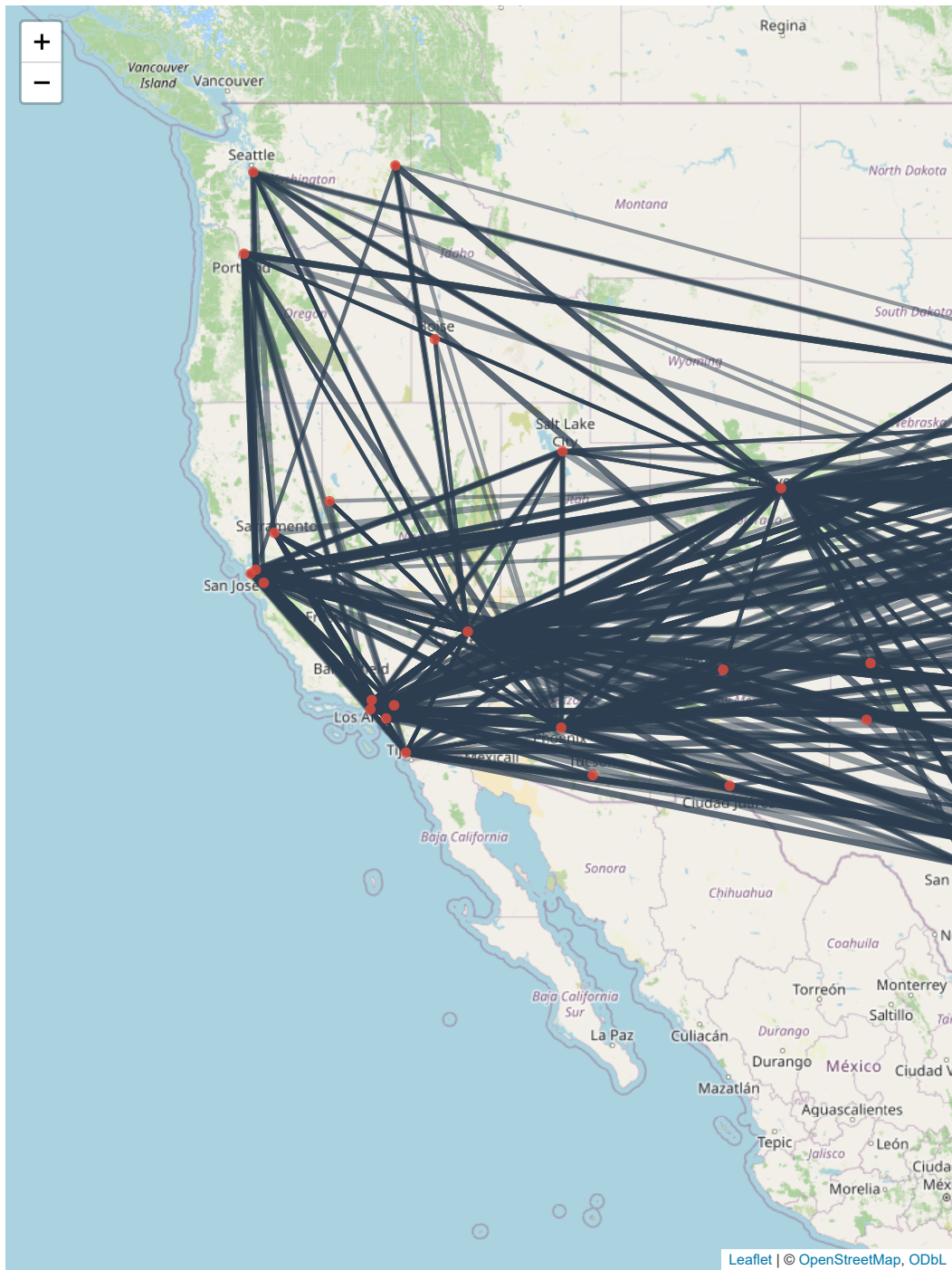
```

# argumento arquivo será o nome do arquivo com caminho completo
arquivo_zip_voos <- file.path(getwd(), "flights.csv.zip")

# chamando a função
resultado_teste <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = arquivo_zip_voos
)

# retornando o mapa
resultado_teste$grafico

```



```
# retornando apenas as primeiras linhas da tabela
head(resultado_teste$tabela)
```

```
# A tibble: 6 x 31
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER ORIGIN_AIRPORT
  <dbl> <dbl> <dbl>      <dbl> <chr>          <dbl> <chr>      <chr>
1  2015     1     1          4 WN             1762 N431WN     DCA
2  2015     1     1          4 WN             3564 N431WN     HOU
3  2015     1     1          4 WN             3564 N431WN     BOS
4  2015     1     1          4 WN             3564 N431WN     BWI
5  2015     1     2          5 WN             1793 N431WN     MCI
6  2015     1     2          5 WN             1793 N431WN     LAX
# i 23 more variables: DESTINATION_AIRPORT <chr>, SCHEDULED_DEPARTURE <chr>,
# DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>, TAXI_OUT <dbl>,
# WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>, AIR_TIME <dbl>,
# DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>, SCHEDULED_ARRIVAL <chr>,
# ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>, DIVERTED <dbl>, CANCELLED <dbl>,
# CANCELLATION_REASON <chr>, AIR_SYSTEM_DELAY <dbl>, SECURITY_DELAY <dbl>,
# AIRLINE_DELAY <dbl>, LATE_AIRCRAFT_DELAY <dbl>, WEATHER_DELAY <dbl>
```

```
lubridate::now(tzone = "America/Sao_Paulo")
```

```
[1] "2026-09-17 00:08:12 -03"
```